

1. Motivation/ Problems solved

Finding the right place to live is one of the most important decisions a person can make, yet it is often overwhelming. Prospective residents and homebuyers must manually research and cross-reference multiple factors — housing prices, neighborhood safety, school quality, and local income levels — across dozens of websites and datasets.

Our application solves this problem by integrating four real-world datasets (IRS income records, school information, and real estate listings) into a single platform. Users can search by state or city and instantly receive a comprehensive neighborhood profile, including average household income, education quality, and housing affordability. This allows users to make data-driven decisions about where to live, invest, or raise a family — all in one place.

2. Features we will definitely implement

Our application provides a location-based exploration platform that allows users to analyze and compare different areas based on income, housing prices, and education availability.

We will implement the following core features:

- Filter locations by income range (e.g., average income greater than a user-defined threshold)
- Filter locations by average housing price
- Filter locations by city or state
- Display aggregated information at the zip code level, including average income, average housing price, and number of schools
- Sort and rank locations based on different criteria (e.g., highest income, lowest housing price, best value ratio)
- View detailed information for each selected zip code

3. Features we might implement (if time permits)

If time allows, we plan to extend the application with more advanced analytical and user-interaction features:

- A ranking system that highlights the best locations based on a composite score (e.g., income-to-housing-price ratio)
- A comparison feature that allows users to compare multiple zip codes side-by-side
- Interactive data visualizations (e.g., charts showing income vs. housing price trends)
- A simple recommendation system that suggests locations based on user preferences
- A bookmarking feature that allows users to save and revisit selected locations
- Map-based visualization to display geographic distribution of results

4. List of pages and their functionality

The application will consist of two main pages with distinct functionality:

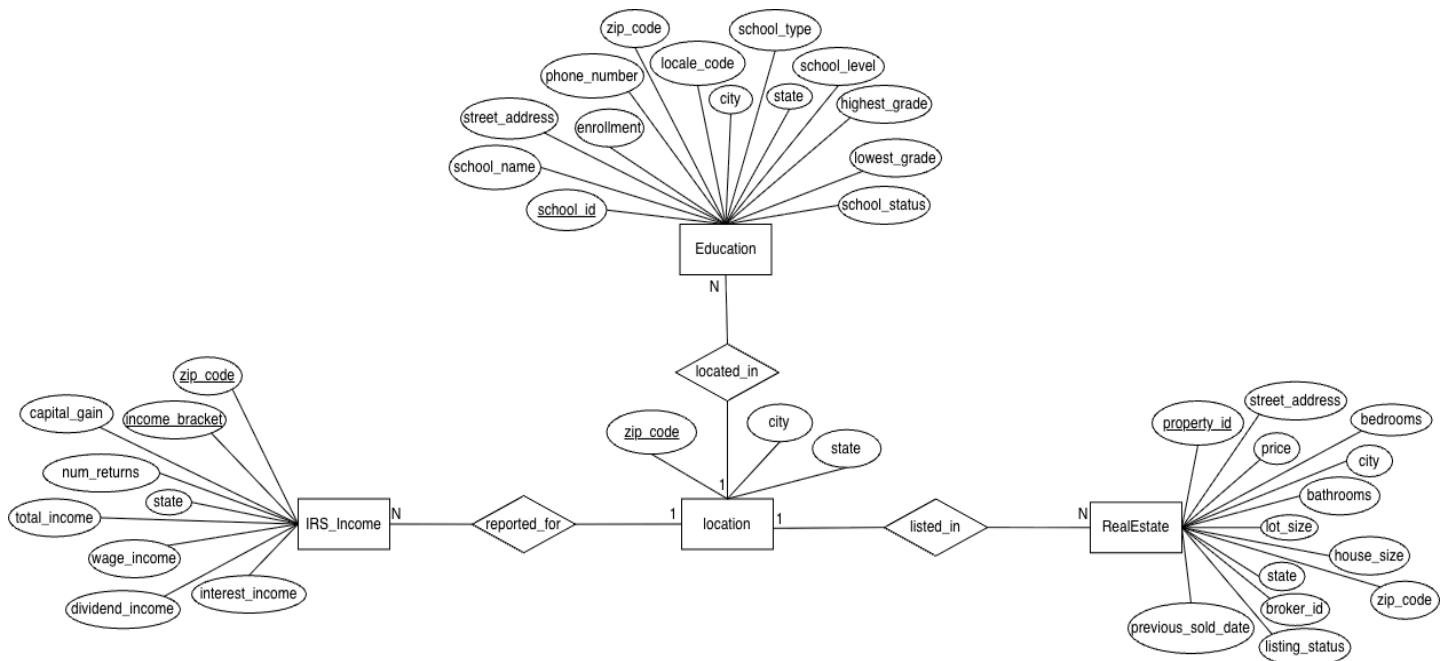
- Search & Results Page
This is the main interface where users can apply filters such as income range, housing

price, and location (city/state). The results are displayed as ranked cards, where each card represents a zip code and shows aggregated information including average income, housing price, and number of schools. Users can explore and compare different areas through this interactive filtering system.

- Zip Code Detail Page

This page is accessed by clicking on a result card. It provides more detailed information for a selected zip code, including deeper insights into income distribution, housing statistics, and education-related data. This page allows users to further investigate a specific area beyond the summary shown on the main page.

5. ER diagram



6. SQL DDL for creating the database

```
CREATE TABLE Location (
    zip_code VARCHAR(5) PRIMARY KEY,
    city TEXT,
    state CHAR(2)
);
```

```

CREATE TABLE Education (
    school_id VARCHAR(20) PRIMARY KEY,
    school_name TEXT NOT NULL,
    street_address TEXT,
    city TEXT,
    state CHAR(2),
    zip_code VARCHAR(5),
    mzip4 VARCHAR(4),
    enrollment INT CHECK (enrollment >= 0),
    phone_number VARCHAR(20),
    locale_code VARCHAR(20),
    school_type TEXT,
    school_level TEXT,
    lowest_grade_taught VARCHAR(20),
    highest_grade_taught VARCHAR(20),
    status TEXT,
    CONSTRAINT fk_education_zip
        FOREIGN KEY (zip_code) REFERENCES Location(zip_code)
);

```

```

CREATE TABLE IRS_Income (
    zip_code VARCHAR(5) NOT NULL,
    state CHAR(2) NOT NULL,
    income_bracket INT NOT NULL,
    num_returns INT CHECK (num_returns >= 0),
    total_income NUMERIC(15,2) CHECK (total_income >= 0),
    wage_income NUMERIC(15,2) CHECK (wage_income >= 0),
    interest_income NUMERIC(15,2) CHECK (interest_income >= 0),
    dividend_income NUMERIC(15,2) CHECK (dividend_income >= 0),
    capital_gain NUMERIC(15,2) CHECK (capital_gain >= 0),

    PRIMARY KEY (zip_code, state, income_bracket)
);

```

```

CREATE TABLE Real_Estate (
    property_id INT PRIMARY KEY,
    street TEXT NOT NULL,
    zip_code VARCHAR(5) NOT NULL,
    city TEXT,
    state CHAR(2),
    price NUMERIC(14,2) CHECK (price >= 0),
    bed NUMERIC(4,1) CHECK (bed >= 0),
    bath NUMERIC(4,1) CHECK (bath >= 0),
    acre_lot NUMERIC(12,4) CHECK (acre_lot >= 0),
    house_size INT CHECK (house_size >= 0),
    status TEXT,
    brokered_by TEXT,
    prev_sold_date DATE,
    CONSTRAINT fk_real_estate_zip
        FOREIGN KEY (zip_code) REFERENCES Location(zip_code)
);

```

7. Data cleaning and pre-processing

Our data cleaning process will mainly focus on handling missing values in the datasets by identifying null entries and deciding whether to remove or adjust affected rows and columns when necessary.

We will also rename certain technical or unclear column names to make them more understandable and consistent across the datasets.

For the Real_Estate dataset, we will convert state names from full names to standard two-letter abbreviations so that they match the format used in the other two datasets. These cleaning steps will help improve data consistency and make integration across tables easier.

8. List of technologies

Our project will use PostgreSQL as the main relational database system for storing and querying property, income, education, and location data. Python with Pandas will be used for data cleaning, preprocessing, and preparing the datasets before importing them into the database. For backend development, we plan to use FastAPI together with Python, while Node.js may be used to support parts of the web application workflow and server-side

integration. The frontend will be built with HTML, CSS, and JavaScript to provide a user-friendly interface for searching and filtering housing data. We will also use GitHub for version control, collaboration, and repository management.

9. Description of what each group member will be responsible for

Frontend: Boming Zhang

Backend: Chiachan Ho

Database Design: Jiajun Shen

Query Optimization: Qinhan Jia